Reference tables from the developer guide

SAIL-PROXY Developer Guide - Chapter 14

What Reaches Users When

Artifact	Updated by	Users receive it
npm `@st-gr/sail-proxy`	`pnpm publish:npm` (step 4 of release)	`npm install -g @st-gr/sail-proxy@latest`
Docker images (`ghcr.io/st-gr/sail-proxy-*`)	`pnpm docker:push` (step 5)	`pnpm docker:pull` / redeploy
Bundled gateway inside the npm package	Rebuilt automatically by `prepublishOnly` → `bundle:gateway`	With the npm package
`api_config.json` for **existing** standalone installs	**Never automatically** — the template is copied once at setup	User re-runs setup or hand-merges new keys (e.g. `supported_beta_headers`)
`api_config.json` for **existing** distributed installs	**Never automatically** — the admin-activated configuration REPLACES the file config wholesale (no merge)	Admin activates a configuration containing the new keys

Added	Effect if the activated configuration predates it
`responsesWebSearchPlugin` hook entries	a hosted `web_search` tool reaches the deployment unrewritten and is rejected with `400 … tools are not allowed for model`
`responsesNamespaceToolsPlugin` hook entries	Codex's `namespace` sub-agent wrapper is rejected the same way, and clients need `--disable multi_agent` again
`capabilities.web_search.max_searches_per_request`	none — absent falls back to the built-in default of 3
`capabilities.namespace_tools.mode`	none — absent falls back to the built-in default of `flatten`

Deployment shape	What an existing install gets	How an operator changes it
Standalone (npm)	keeps `true` — the template is copied **once**, at first run, and never re-merged	edit `~/.sail-proxy/api_config.json`
Docker	keeps `true` — the admin-activated configuration **replaces** the file configuration wholesale, with no merge, so a new template key is invisible until it is in an activated configuration	activate a configuration containing the new value, in the cockpit
Kyma	keeps `true`, for the same reason	activate a configuration containing the new value, in the cockpit
Any install predating the `file_search` block	gets `false`, from `FILE_SEARCH_DEFAULTS`	add the key to opt back in

SAIL-PROXY Developer Guide - Chapter 15

`reranker_search_events` — one row per search

Column	Meaning
`id`	`rse_…` primary key; `reranker_candidate_labels.event_id` references it `ON DELETE CASCADE`.
`created_at`	Insert time.
`query_text`, `query_hash`	The caller's original query, and its sha256. The hash is what you group by to find repeated queries without reading them.
`source`	Free-text tag from `capabilities.file_search.teacher_logging.source` — how you separate `production` traffic from an offline harvesting run in the same table.
`store_ids`, `owner_email`	Which vector stores were searched, and by whom.
`retrieval_mode`	`reranked` or `rrf_only`.
`reranker_available`	`false` for an `rrf_only` search. **Rows with `reranker_available = false` carry no teacher signal** — they have retrieval ranks but every `teacher_rank` is NULL. Filter them out of any training export.
`candidates_requested` / `candidates_returned`	The configured recall width (`capabilities.file_search.hybrid.candidates`) vs. how many chunks recall actually produced and handed to the teacher. `candidates_returned` is the number of label rows this event has.
`rrf_k`, `lexical_enabled`, `embedding_model`, `embedding_dim`	The retrieval configuration in force. Retrieval config drifts; a dataset that does not record it cannot be pooled across time.
`reranker_provider`, `reranker_model`	Which teacher produced the labels. NULL when none did.
`reranker_search_units`	Billed search units reported by the reranker deployment — the cost side of the "is the teacher worth it?" question.
`rewrite_used`, `query_rewritten`	Whether the LLM query rewriter fired, and what it produced. The *rewritten* query is what was actually embedded and matched.
`embed_latency_ms`, `rerank_latency_ms`, `total_latency_ms`	Measured with a monotonic clock (`process.hrtime.bigint()`), not wall-clock. `rerank_latency_ms` is NULL for `rrf_only`.
`top_k`, `score_threshold`	The truncation the caller asked for — the reason a chunk can be highly ranked by the teacher and still not `selected`.
`request_id`, `metadata`	Correlation id and a free `jsonb` slot.

`reranker_candidate_labels` — one row per candidate of one search

Column	Meaning
`id`, `event_id`	Primary key and the owning event; `UNIQUE (event_id, candidate_index)`.
`candidate_index`	0-based position within the array handed to the teacher. This is the join key between an event's candidates and the teacher's output.
`store_id`, `file_id`, `ord`, `filename`	Identifies the chunk.
`chunk_hash`	sha256 of the chunk text — **always written**, whether or not the text itself is.
`chunk_text`	The verbatim chunk text, or NULL. See §4.
`chunk_tokens`	Reserved; **always NULL today**. Nothing on the write path populates it — the chunk's token count is not carried through recall. Do not filter or aggregate on it.
`retrieval_rank`	1-based rank in the fused RRF ordering, i.e. what retrieval alone would have returned.
`rrf_score`	The fused RRF score: the sum of `1/(rrf_k + rank)` over the arms that recalled this chunk. Not a similarity and not calibrated.
`vector_rank`, `vector_score`	The vector arm's rank and its raw **cosine distance** (lower is better, never inverted). NULL when only the lexical arm recalled this chunk.
`lexical_rank`, `lexical_score`	The lexical arm's rank and raw `ts_rank` (higher is better). NULL when only the vector arm recalled it, or when `capabilities.file_search.hybrid.lexical_enabled` is false.
`teacher_rank`	**The training signal.** 1-based position in the teacher's relevance-descending output. NULL on an `rrf_only` event.
`teacher_score`	The teacher's raw relevance score. Auxiliary, uncalibrated — see §2.

Column	Meaning
`selected`	Whether this chunk survived `score_threshold` and `top_k` and was actually served to the caller.
`attributes`	The file's attribute `jsonb`, as it was at search time.

3. Enabling collection

Key	Default	Effect
`enabled`	`false`	Master switch. While false the search path writes nothing at all.
`store_chunk_text`	`false`	Whether `chunk_text` is persisted verbatim. `chunk_hash` is written either way.
`sample_rate`	`1.0`	Fraction of searches recorded, sampled per search. Lower it on a high-traffic gateway; each event costs one transaction and `candidates_returned + 1` rows.
`source`	`"production"`	Tag written to every event, for separating collection runs.
`max_concurrent_writes`	2	Ceiling on in-flight label writes. Beyond it, writes are **dropped, not queued** — see §5. Validated range 1–10.

SAIL-PROXY Developer Guide - Chapter 3

Google Gemini route

File	Role
`routes/googleRoutes.ts`	mounts `/google/v1beta` and `/google/v1`; auth, service auth, quota middleware
`controllers/googleController.ts`	parses the request, resolves the route (`chooseRoute`), entitlement, hooks, error shaping
`controllers/googleDispatch.ts`	the three transports: native deployment, orchestration chat bridge, embeddings
`controllers/googleWire.ts`	conventions shared by controller and dispatch: error envelope, SSE headers/teardown, usage folding
`services/googleGeminiService.ts`	`parseModelMethod`, `resolveGeminiDeployment`, `geminiUrl`, usage/error helpers
`google/orchestrationBridge/requestTranslator.ts`	Gemini body → `SapV2CompletionRequest`
`google/orchestrationBridge/geminiParts.ts`	one Gemini `Part` → one orchestration content block; `UnsupportedGeminiInputError`
`google/orchestrationBridge/responseTranslator.ts`	orchestration envelope → Gemini response (+ `usageMetadata`)
`google/orchestrationBridge/streamTranslator.ts`	`StreamChunk` → Gemini SSE frames (+ final usage)
`utils/geminiBodyAdapter.ts`	text extraction/replacement for the pseudonymization plugin

Method	Condition (checked in order)	Route
`generateContent` / `streamGenerateContent`	explicit `<m>--deployed`, deployment's provider matches `/google|gemini/i`	**native**, on that deployment
	explicit `<m>--deployed`, deployment's provider is NOT Google (e.g. a Claude deployment asked for in Gemini shape)	**bridge**, on the base model name
	bare `<m>` with a Google-provider twin (`resolveGeminiDeployment`)	**native**, on the twin
	bare `<m>`, `getModelDetails(model)` returns an entry	**bridge**, `modelName = details.model || baseModel`
	none of the above	404

Method	Condition (checked in order)	Route
`embedContent`	bare `<m>`'s details carry `allowedScenarios` including `orchestration`	**embeddings via orchestration** (exact usage)
	else a Google-provider twin or explicit `--deployed` id whose capabilities include `embedding`	**embeddings native** (estimated usage)
	else	404

SAP-RPT tabular prediction route

File	Role
`routes/sapRptRoutes.ts`	mounts `POST /:model/predict` and `POST /:model/predict-parquet`; `shapeMiddlewareErrors` first, then `createUnifiedTokenAuth()`, then `quotaEnforcement` — no hook middleware, no tool governance
`controllers/sapRptController.ts`	resolves the model, checks entitlement, forwards the body verbatim, folds usage, relays SAP's response
`sapRpt/usage.ts`	`cellsFromResponse` (cells from the response's `metadata`), `accountedModel` (the `--deep-context` id)
`sapRpt/errors.ts`	`rptError` — the gateway's own refusals in SAP's `{detail: [{loc,msg,type}], status:{code,message}}` shape
`sapRpt/shapeMiddlewareErrors.ts`	reshapes `createUnifiedTokenAuth()`'s 401 and `quotaEnforcement`'s 429 from the OpenAI `{error:{...}}` envelope into the SAP shape above, so a client of this route parses one error shape throughout
`utils/deployedTwin.ts`	`resolveDeployedTwin` — the bare-model / `--deployed` twin resolver this route shares with `/google` and the OpenAI Realtime route

OpenAI Realtime route

File	Role
`realtime/realtimeUpgrade.ts`	path match, admission chain, twin resolution, entitlement, upstream connect, client handshake, observer wiring; `RealtimeDeps` for tests
`realtime/admission.ts`	`prepareUpgradeRequest` (what the Express middlewares read), the recording response shim, `runMiddleware`, `writeRefusal` (a recorded refusal as one raw HTTP/1.1 response)
`realtime/relay.ts`	frames forwarded unchanged both ways; observer hook after forwarding; close propagation
`realtime/realtimeObserver.ts`	pure: `classifyFrame`, `usageMetricsFromResponseDone`
`realtime/closeCodes.ts`	pure: close codes/reasons, `propagatedClose` (1005 → 1000, 1006 → 1011)
`utils/deployedTwin.ts`	`resolveDeployedTwin` — the bare-model / `--deployed` twin resolver shared with `/google`

docs/developer/tool-governance.md

Identity and evaluation

File	Responsibility
`identity.ts`	`ToolIdentity` (a plain `namespace:name` string), th five constructors (`functionTool`, `hostedTool`, `hostedNamedTool`, `mcpServer`, `mcpTool`), `toolName`/`resolveInvoked` (reconciling an invoked function call with the tool that was declared) `PATTERN_SYNTAX` (`^(function|hosted|mcp):([^*\s]+\*?|\*)$` — the corrected regex that also accepts a bare namespace wildcard such as `function:*`, matched by the admin's own copy in

File	Responsibility
	`toolPolicyService.ts`), `matches`/`matchesAny` (exact match, or prefix match when the pattern ends in `*`)
`stripNotice.ts`	the line strip mode adds to the request naming the removed tools; the text is invariant for a given set of to so a client that is stripped on every turn keeps its prom cache, and the `[tool policy]` marker keeps a repla body from accumulating notes
`evaluate.ts`	Pure evaluation: `deniedBy(block, identity)` (de always wins; an empty allow list allows everything not denied); `evaluate(declared, user, key, for` merges a user policy and an optional key policy — mo restrictive mode wins (`MODE_RANK`: monitor 0 < strip 1 reject 2), an identity is denied if either block denies it, a under `strip` a `forced` tool (from `tool_choice`) tha lands in the denied set flips the whole result to `rejec` instead of silently forcing a tool that was just stripped
`adapters/{anthropic,openaiChat,responses,gemini}.ts`	One per API family, each a pure `ToolAdapter` (`adapters/types.ts`): `declaredTools`, `forcedT` `stripTools` (returns a new body, never mutates), `invokedTools` (non-streaming), `invokedToolsFromChunk`/`invokedToolsFromSt` (SSE), `rejectionBody`. The Responses adapter's `stripTools` narrows an MCP tool's `allowed_tool` rather than dropping the whole `mcp` entry when only so of its tools are denied
`middleware.ts`	`toolGovernance(adapter)`, mounted per route; `policyBlocksFromRequest` (NO admin exemption tool policy is a security control and binds the holder of account or key, unlike the model entitlement check)
`record.ts`	`req.toolGovernance` state, `recordInvokedTool` `toolsForEvent` (declared entries first, then invoked, each carrying the identity's decision)

Mode	Every scoped allow is a literal name	One scoped allow has a trailing `*`
monitor	`mcp:S` decision `monitored`, `narrow` untouched	same
strip	`narrow.set(S, names)` — the intersection of every block's names — decision `stripped`	decision `monitored`, no `narrow` entry (a prefix cannot be rendered as names)
reject	`mcp:S` pushed onto `denied`, the whole request refused	same

Trust chain (2026-09-22 §3)

Family	Result	Paired with
OpenAI chat (`openaiChat.ts`)	`role:"tool"` message, `tool_call_id`; a legacy `role:"function"` message names its own tool	assistant `tool_calls[].function.name`
Responses (`responses.ts`)	`function_call_output` / `custom_tool_call_output`, `call_id`; `mcp_call` items that carry `output` and any other `<type>_call` item count directly, mapped through `HOSTED_CALL_TYPES` (`computer_call` → `hosted:computer_use_preview`) the same table `outputItemIdentity` uses for invocations	the matching `function_call` / `custom_tool_call` input item
Anthropic (`anthropic.ts`)	`tool_result` / `mcp_tool_result` block, `tool_use_id`; any other `*_tool_result` block (`web_search_tool_result`) counts directly as its hosted tool with the suffix stripped	the `tool_use` / `mcp_tool_use` block's `name` (and `server_name` for the MCP case)
Gemini (`gemini.ts`)	`functionResponse` part	its own `name` (Gemini declares no hosted or MCP tool shape, so this is the adapter's whole `resultSources`)